

Start Monitoring — End-to-End Flow

DeepHorizon Security · Mobile ↔ rewp2 ↔ SOC · Technical reference

0. The actors

Layer	Path	Role
Mobile UI	src/screens/HomeScreen.tsx	"Start Monitoring" button
Mobile state	src/features/monitoring/MonitoringSession.tsx	Provider — owns session lifecycle
Mobile BG task	src/features/monitoring/backgroundLocation.ts	TaskManager.defineTask — OS-delivered GPS
Mobile tier engine	src/features/monitoring/tierSignal.ts	Local T1→T2→T3 decisions
Mobile check-in	src/features/monitoring/CheckInWatcher.tsx	Foreground tick + expo-notifications
Mobile API	src/api/monitoring.ts	Thin HTTP wrappers
rewp2 – tier brain	rewp2/src/routes/	entry.ts , checkin.ts , escalation.ts , tracking.ts
rewp2 – SOC	rewp2/src/services/soc.dispatch.service.ts	Outbox + fanout
location_service	backend2	Stateless GPS recorder
new-dashboard	SOC Next.js app	Consumer of soc:event

1. The tap — startSession()

User taps "Start Monitoring" in HomeScreen.tsx :

```
const monitoring = useMonitoringSession();
monitoring.startSession({ tripType: 'cab' });
```

Phase A — synchronous (returns instantly, navigates to QuickStart)

#	What happens
1	Reset refs: sequenceRef=0 , sessionIdRef='sm_{timestamp}' , tierRef=1 , destinationRef=null , autoStoppedRef=false
2	Clear stale distance from prior session (SecureStore.deleteItemAsync on distanceM , lastLat , lastLng)
3	Write lastCheckinAt to SecureStore (seed for check-in anchor)
4	Spawn new TierSignalService() → store in tierServiceRef
5	Register tierService.onTierChange() — on tier flip: recompute nextCheckinAt , call reconfigureForTier()
6	Compute t1DueIso = startMs + TIER_INTERVAL_MIN[1]
7	setState({ isActive: true, startedAt, tier: 1, nextCheckinAt: t1DueIso, ... })

Phase B — deferred async (doesn't block UI)

#	Call	What it does
1	<code>Promise.all([SecureStore.setItemAsync(...)])</code>	Writes <code>active='true'</code> , <code>userId</code> , <code>sessionId</code> , <code>startedAt</code> , <code>tripType</code> , <code>appState</code> , <code>sequence=0</code> — the bridge to the background task
2	<code>entryStart(userId)</code>	<code>POST /handling/entry</code> → rewp2 creates <code>sessionStore[userId]</code> , calls <code>location_service/tracking/start</code>
3	<code>Location.getCurrentPositionAsync({ Balanced })</code>	Seed GPS fix, stored in <code>latestLocationRef</code>
4	<code>sendPingFromLatestRef.current?.()</code>	Fires immediate first ping so <code>/users/active</code> populates fast
5	<code>restartBackgroundForTier(1)</code>	Registers the background location task (§3)
6	<code>startWatcherForTier(1)</code>	Starts foreground GPS watcher (§2)
7	<code>Location.requestBackgroundPermissionsAsync()</code>	Asks for <code>ACCESS_BACKGROUND_LOCATION</code>

2. Foreground flow (app is open)

The GPS watcher

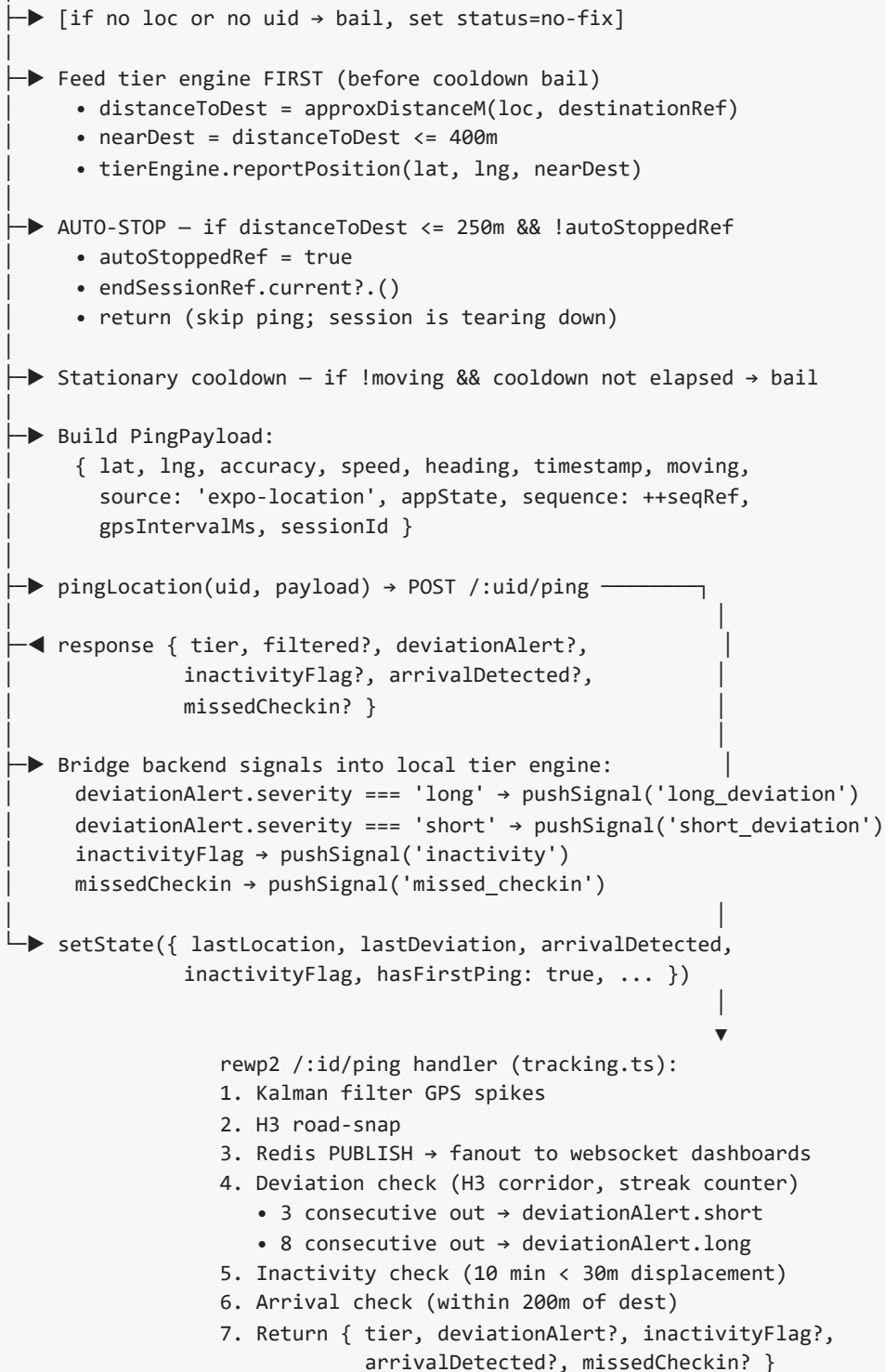
`startWatcherForTier(tier)` calls `Location.watchPositionAsync` with per-tier params:

```
TIER_PROFILES[1] = { accuracy: Lowest,    pingMs: 15000, distanceM: 10, stationaryCooldownMs: 60000 }  
TIER_PROFILES[2] = { accuracy: Balanced,  pingMs: 10000, distanceM:  8, stationaryCooldownMs: 30000 }  
TIER_PROFILES[3] = { accuracy: High,      pingMs:  5000, distanceM:  4, stationaryCooldownMs:  5000 }
```

Every watcher callback stores in `latestLocationRef` and schedules `sendPingFromLatest()` via `pingTimerRef`.

sendPingFromLatest() — foreground ping loop

sendPingFromLatest() – runs every T1=15s / T2=10s / T3=5s



Polyline + remaining distance

Independent of pings, once a destination is set, `remainingTimerRef` polls `GET /destination/:uid/remaining` every 10 s → drives the Route tile.

3. Background flow (app swiped / killed / backgrounded)

Registration

```
await Location.startLocationUpdatesAsync(MONITORING_BG_TASK, {
  accuracy: BG_ACCURACY_BY_TIER[tier], // T1=Lowest, T2=Balanced, T3=High
  interval: BG_INTERVAL_BY_TIER[tier], // T1=60s, T2=15s, T3=5s
  distanceInterval: 5,
  foregroundService: {
    notificationTitle: `Deep Horizon · Monitoring · T${tier}`,
    notificationBody: '...',
    notificationColor: '#1d6fa4',
  },
  pausesUpdatesAutomatically: false,
});
```

This spawns an Android foreground service with a persistent notification — that's what keeps the process alive when the app is backgrounded. AndroidManifest sets `android:stopWithTask="false"` on `LocationTaskService`, so even swipe-from-Recents leaves the service running.

Task definition — `TaskManager.defineTask(MONITORING_BG_TASK, ...)`

Runs in an **isolated JS context** (no React, no state, no navigation):

```
OS delivers batched locations → backgroundTaskCallback({ data: { locations }, error })
├─▶ Read gate from SecureStore: BG_KEYS.active === 'true' ? (else no-op)
├─▶ Read bridge: userId, sessionId, appState, sequence
├─▶ For each location (sorted chronologically):
│   1. sequence++
│   2. updateTraveledDistance(lat, lng, accuracy)
│   3. pingLocation(userId, {
│       ...payload,
│       source: 'background_task', ← distinguishable
│       appState,
│       sequence,
│     })
└─▶ SecureStore.setItemAsync(BG_KEYS.sequence, String(sequence))
```

Key fact: the background task uses the *same* `POST /:uid/ping` endpoint as the foreground. The only difference is the `source` and `appState` fields in the payload.

Why it works on stock Android and fails on ColorOS

OS	Behavior
Stock Android, Pixel	Foreground service keeps process alive indefinitely. Pings continue.
Most OEMs	Same, with minor Doze coalescing below 15 s cadence.
ColorOS (Oppo/Realme/OnePlus)	Kills process on swipe despite <code>stopWithTask="false"</code> . Requires user to enable "Allow background activity" + pin in Recents.

Rehydration on cold restart

BootGuard finishes → userId set → rehydrate effect in MonitoringSession.tsx:

1. Read BG_KEYS.active, userId, sessionId, startedAt, tripType
2. If active==='true' AND Location.hasStartedLocationUpdatesAsync(MONITORING_BG_TASK)===true
3. Read nextCheckinAt, lastCheckinAt, destinationLat/Lng/Name
4. Spawn new TierSignalService, restore lastCheckinAtRef, destinationRef
5. setState({ isActive: true, destination: restored, nextCheckinAt, ... })
6. startWatcherForTier(1) (for foreground UI – BG is already running)
7. If destination restored → re-fetch polyline via GET /destination/:uid,
re-arm remaining poll

4. Check-in flow

Countdown source

`state.nextCheckinAt` (ISO string) is the single source of truth. Computed from:

- Session start: `startedAt + TIER_INTERVAL_MIN[1]`
- Tier shift: `lastCheckinAt + TIER_INTERVAL_MIN[newTier]`
- Successful response: advances via `applyCheckinUpdate`

CheckInWatcher.tsx — mounted once at top of nav tree

Path A — Foreground tick (every 1 s):

```
if (isActive && !inEscalation):  
  if now > dueMs + 30s: pushSignal('missed_checkin') + navigate Escalation  
  elif now > dueMs:    navigate CheckInPrompt (modal with 30s countdown)
```

Path B — Background notifications (AlarmManager via expo-notifications):

```
On every nextCheckinAt change:  
  cancelPending()  
  schedule prompt notification at dueDate (channelId='monitoring')  
  schedule missed notification at dueDate + 30s
```

Tap listener (`Notifications.addNotificationResponseReceivedListener`):

- `type === 'monitoring_checkin'` → navigate to `CheckInPrompt`
- `type === 'monitoring_checkin_missed'` → `pushSignal('missed_checkin')` + navigate Escalation

User responds "Safe"

`CheckInPromptScreen` → `checkinRespond(uid, true)` → `POST /handling/checkin/:uid/respond`:

```
rewp2:  
  session.last_response = 'safe'  
  if tier > 1: shiftTier(session, 1, 'user_confirmed_safe')  
  session.next_checkin_at = now + interval  
  response: { tier: 1, next_checkin_at, ... }
```

User responds "I Need Help" OR misses window

- "Need Help" → `POST /handling/checkin/:uid/respond { is_safe: false }` → `shiftTier(3, 'user_needs_help')` + `dispatchToSoc('user_needs_help')`
- Missed → `POST /handling/checkin/:uid/missed` → `shiftTier(3, 'missed_checkin')` + `dispatchToSoc('checkin_missed')` + `trigger_escalation: true` in response

5. Tier engine — `tierSignal.ts`

Local, client-side, drives GPS strategy without waiting for backend.

Signals

```
type SignalType = 'inactivity' | 'short_deviation' | 'long_deviation'  
                | 'missed_checkin' | 'user_needs_help'
```

Rules

```
tier = 3 if active.has('long_deviation') | active.has('missed_checkin') | active.has('user_needs_help')
      = 2 if active.has('short_deviation') | active.has('inactivity')
      = 1 otherwise
```

Inactivity detector

- `INACTIVITY_WINDOW_MS = 10 min`, `INACTIVITY_DISTANCE_M = 30 m`
- Anchor-based: while within 30 m of anchor, dwell; if `now - anchor.since >= 10 min` → push `'inactivity'`
- Suppressed when `nearDest === true` (within 400 m of destination)

On tier change

```
tierService.onTierChange((newTier, reason, prev) => {
  reconfigureForTier(newTier) // restart BG task + foreground watcher with new params
  setState({ tier: newTier, intervalMinutes: TIER_INTERVAL_MIN[newTier], nextCheckinAt: ... })
})
```


6. Auto-stop on arrival (250 m)

In `sendPingFromLatest`:

```
if destinationRef && distanceToDest <= 250 && !autoStoppedRef:
  autoStoppedRef = true
  endSessionRef.current?.()  // latched – won't re-fire
```

Reset on `startSession()` and `setDestination()`. Destination is persisted to SecureStore so this works after cold restart.

7. `endSession()` — teardown

```
stopWatchersInternal()          ← stops FG watcher + ping timer + tier engine
setState({ isActive: false, nextCheckinAt: null, destination: null, ... })
SecureStore.setItemAsync(BG_KEYS.active, 'false') ← gates BG task (silent no-op on delivery)
SecureStore.deleteItemAsync(startedAt, tripType, nextCheckinAt, lastCheckinAt,
                             destinationLat, destinationLng, destinationName)
Location.stopLocationUpdatesAsync(MONITORING_BG_TASK) ← kills the foreground service
entryEnd(userId)                  ← PUT /handling/entry/:uid/end
stopTracking(userId)              ← POST /:uid/stop → saves session + logs
```

8. Full API surface

Mobile → rewp2

Phase	Function (monitoring.ts)	Endpoint	Triggered by
Session start	<code>entryStart</code>	<code>POST /handling/entry</code>	<code>startSession</code> phase B
Session details	<code>entryDetails</code>	<code>PUT /handling/entry/:id/details</code>	Trip-type / contacts pickers
Destination	<code>setDestination</code>	<code>POST /destination/:id/set</code>	"Set destination" UI
Destination	<code>clearDestination</code>	<code>POST /destination/:id/clear</code>	"Clear destination"
Destination	<code>getDestination</code>	<code>GET /destination/:id</code>	Rehydrate / polyline
Destination	<code>getDestinationRemaining</code>	<code>GET /destination/:id/remaining</code>	10 s poll
Pings	<code>pingLocation</code>	<code>POST /:id/ping</code>	FG watcher + BG task
Check-in start	<code>checkinStart</code>	<code>POST /handling/checkin/:id/start</code>	After <code>entryStart</code>
Check-in respond	<code>checkinRespond</code>	<code>POST /handling/checkin/:id/respond</code>	"Safe" / "Help" tap
Check-in missed	<code>checkinMissed</code>	<code>POST /handling/checkin/:id/missed</code>	30 s countdown expiry
Escalation	<code>escalationTrigger</code>	<code>POST /handling/escalation/:id/trigger</code>	Missed / Need help
Escalation	<code>escalationCancel</code>	<code>PUT /handling/escalation/:id/safe</code>	"I'm safe" on Escalation screen
Escalation	<code>escalationStatus</code>	<code>GET /handling/escalation/:id/status</code>	Polled by UI
Session read	<code>getUserStream</code>	<code>GET /user/:id/stream</code>	Map live follow
Session end	<code>stopTracking</code>	<code>POST /:id/stop</code>	<code>endSession</code>
Session end	<code>entryEnd</code>	<code>PUT /handling/entry/:id/end</code>	<code>endSession</code>
Summary	<code>entrySummary</code>	<code>GET /handling/entry/:id/summary</code>	Post-session screen
Events	<code>getSessionEvents</code>	<code>GET /session/:id/events</code>	Timeline

rewp2 → outside

From	To	Purpose
<code>shiftTier()</code>	<code>PUT {LOCATION_SERVICE_URL}/tracking/:uid/tier</code>	Tell location_service about cadence change
<code>/entry</code>	<code>{LOCATION_SERVICE_URL}/tracking/start</code>	Start location_service session
<code>/entry/:id/end</code>	<code>{LOCATION_SERVICE_URL}/tracking/:uid/stop</code>	Stop location_service session
<code>dispatchToSoc()</code>	Socket.IO room <code>soc:dashboard</code> + <code>SOC_WEBHOOK_URL</code>	SOC fanout
<code>POST /:id/ping</code>	Redis PUBLISH <code>location_updates</code>	Dashboard live-follow, Socket.IO

rewp2 internal — SOC dispatch call sites

Call site	Event fired	Severity
<code>escalation.ts /trigger</code>	<code>escalation_triggered</code>	critical
<code>escalation.ts /advance</code>	<code>escalation_advanced</code>	critical
<code>escalation.ts /safe</code>	<code>escalation_resolved</code>	info
<code>checkin.ts shiftTier() when newTier === 3</code>	<code>tier_escalated</code>	critical
<code>checkin.ts /respond is_safe=false</code>	<code>user_needs_help</code>	critical
<code>checkin.ts /missed</code>	<code>checkin_missed</code>	critical

9. Every SecureStore key in the flow

Defined in `backgroundLocation.ts` as `BG_KEYS` :

Key	Type	Written by	Read by
<code>monitoring_bg_active</code>	<code>'true'/'false'</code>	<code>startSession</code> , <code>endSession</code>	BG task gate, rehydrate
<code>monitoring_bg_user_id</code>	string	<code>startSession</code>	BG task, rehydrate
<code>monitoring_bg_session_id</code>	string	<code>startSession</code>	BG task, rehydrate
<code>monitoring_bg_app_state</code>	foreground/background	AppState listener	BG task (for ping payload)
<code>monitoring_bg_sequence</code>	number	BG task (inc), FG reset	BG task
<code>monitoring_bg_started_at</code>	ms epoch	<code>startSession</code>	rehydrate
<code>monitoring_bg_trip_type</code>	cab/meeting/foot/custom	<code>startSession</code>	rehydrate
<code>monitoring_bg_distance_m</code>	number	BG task + FG	Session summary
<code>monitoring_bg_last_lat</code> / <code>lng</code>	number	BG task + FG	<code>updateTraveledDistance</code>
<code>monitoring_bg_next_checkin_at</code>	ISO string	Ping flow, tier change	rehydrate
<code>monitoring_bg_last_checkin_at</code>	ms epoch	Check-in response	rehydrate
<code>monitoring_bg_destination_lat</code> / <code>lng</code> / <code>name</code>	coords + string	<code>setDestination</code>	rehydrate, <code>endSession</code> cleanup

10. One-picture summary

